10 – Inter Process Communication

How processes exchange/share information?

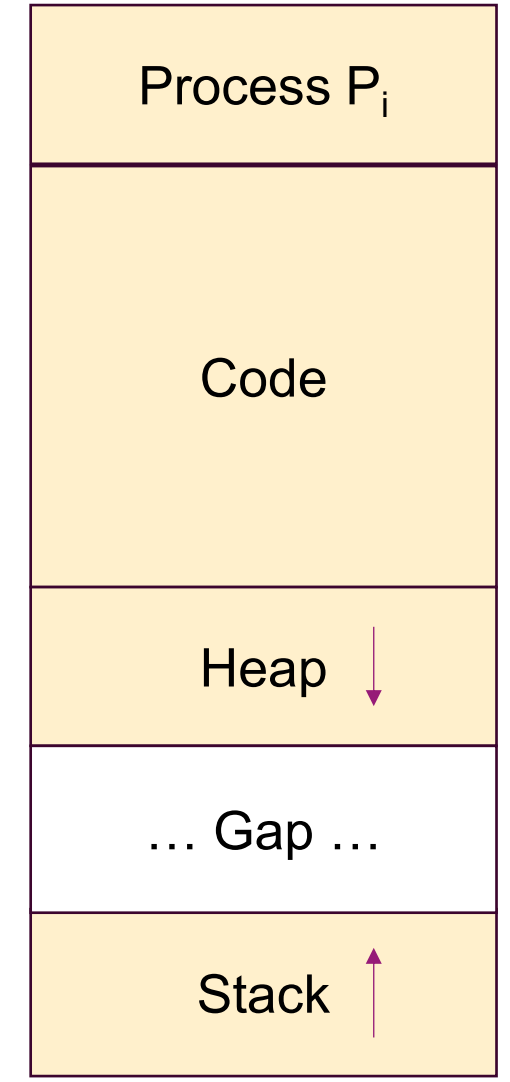
Jérôme Landré – **jerome.landre@ju.se**

OUTLINE

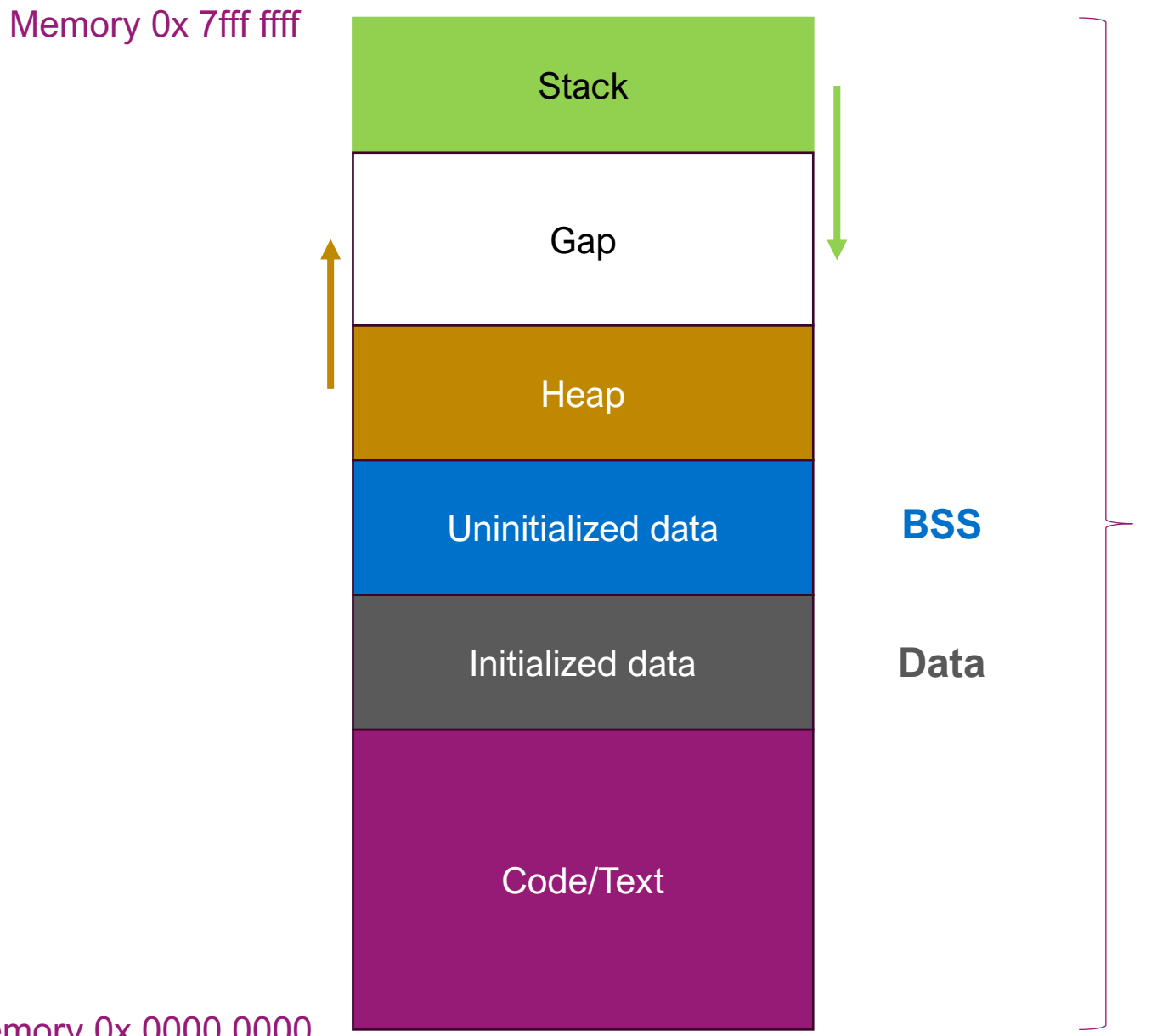
0) Processes and threads
1) Why IPC?
2) IPC methods
3) Signals
4) Pipes
5) Message queues
6) Shared memory
7) Semaphores
8) Sockets

0) PROCESSES AND THREADS

- A **process** is a program that runs on a computer
- A process P_i is composed of:
  - A **code** to execute (machine language)
  - A **heap** to store the variables and objects of the program
  - A **stack** to store the execution environment



Memory zone allocated by the OS

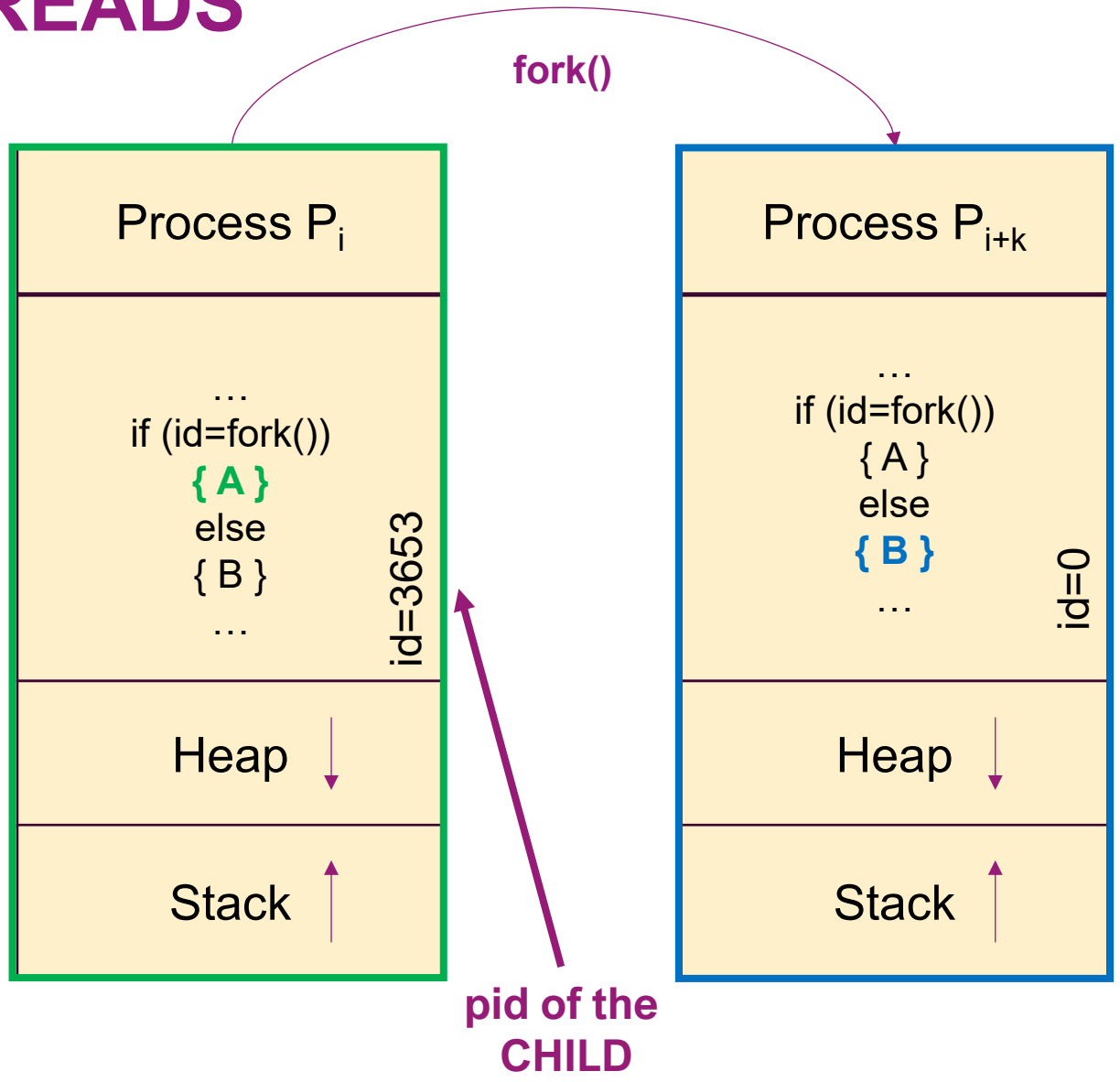


Memory of the process (2 MB)

0) PROCESSES AND THREADS

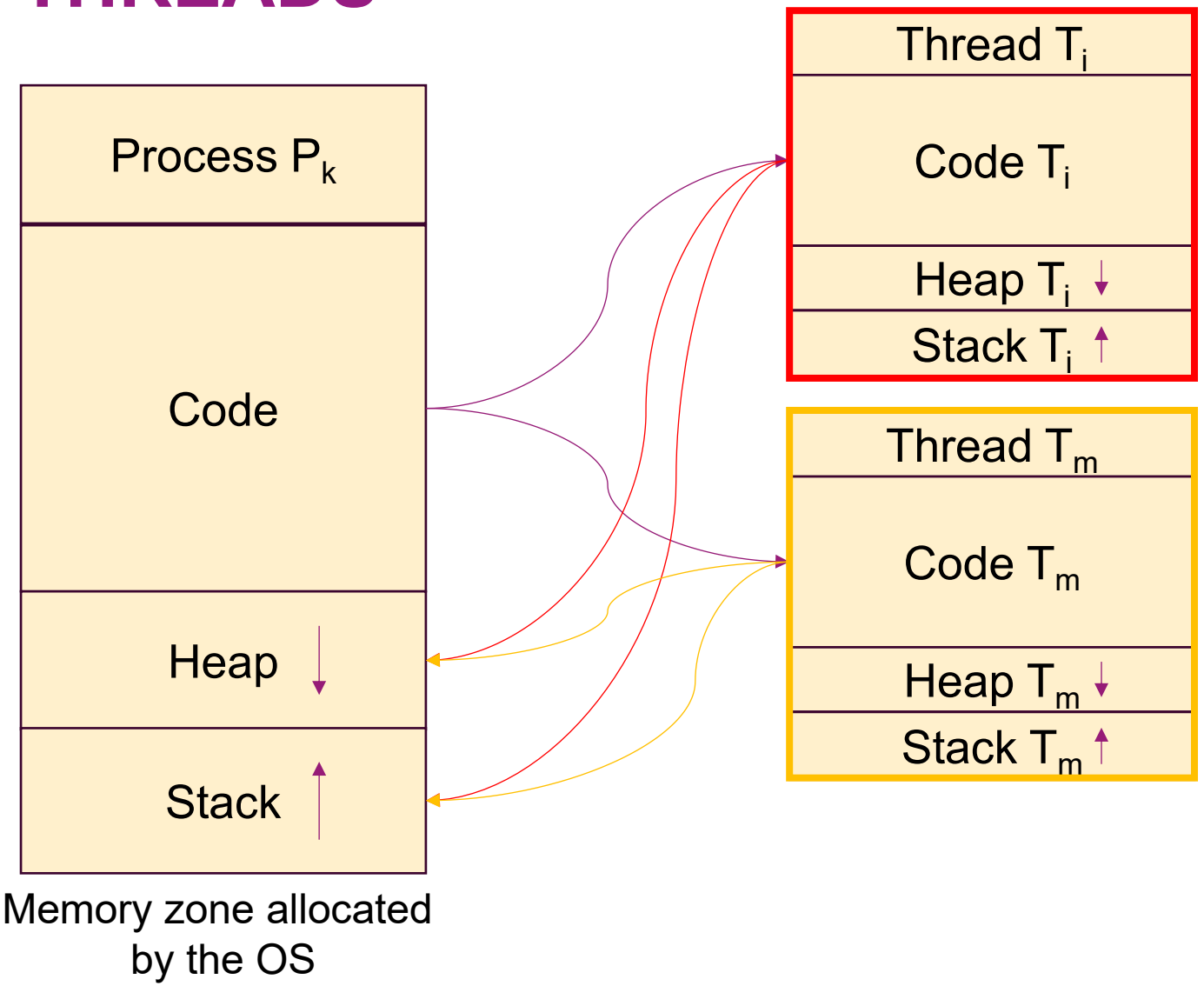
- Creation of a process
  - On Linux, a process is created by running the system function fork()
  - fork() creates two copies of the running process by copying its code

No shared memory, so problem to exchange/share information between processes



0) PROCESSES AND THREADS

- A thread is a light weight code running into a process

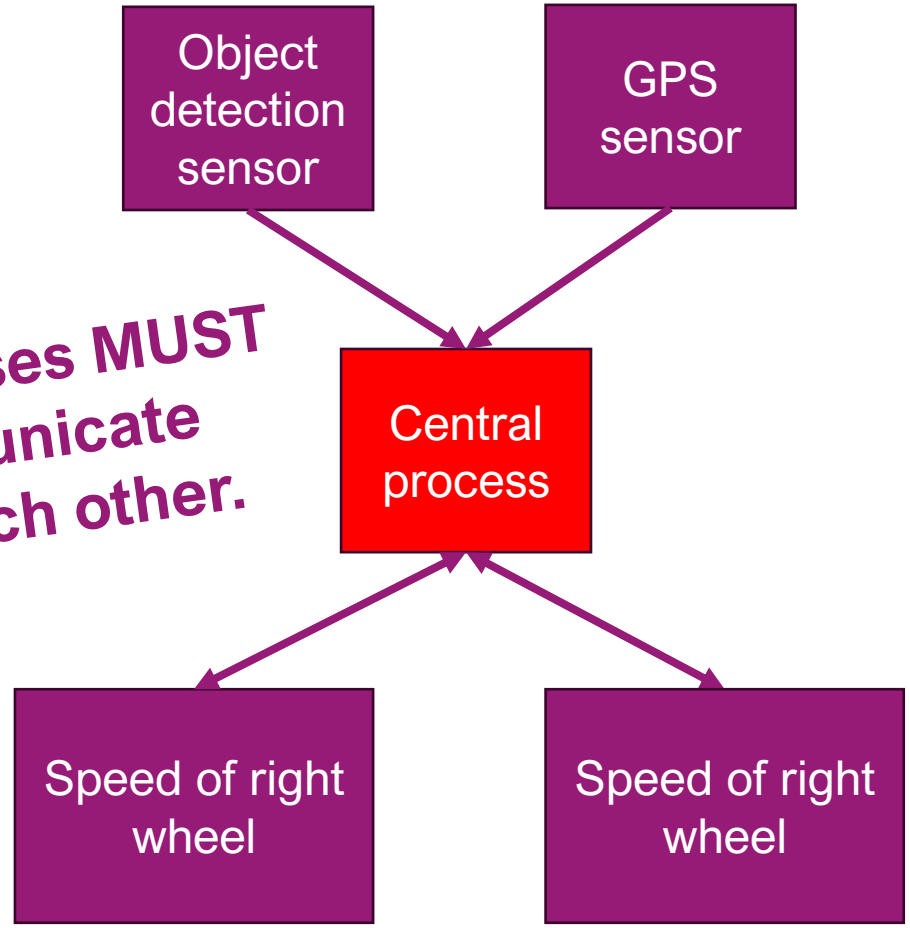


Shared memory, so no problem to exchange/share information between threads

1) WHY IPC?

- Imagine you have an autonomous lawn mower (from a very famous brand near Jönköping for instance). It is composed of:
  - Hardware:
    - Motors/engines/actuators
    - Sensors: Positioning (GPS), Speed, Object/Obstacle detection…
  - Software:
    - A central process that synchronizes/drives all the other processes
    - A process to read the object detection sensor,
    - A process to read the GPS information,
    - A process to read the speed,
    - A process to write the speed to the electric motors (left and right),
    - …

⇒ **processes MUST communicate with each other.**



1) WHY IPC?

- What is IPC?
  - Mechanism for **processes** to **communicate** and **synchronize** their actions.
  - Enables **data sharing**, **resource management**, and **parallel** processing.
- Why is IPC Important?
  - Modern OSes run multiple processes **concurrently**.
  - Processes often need to **share data** or coordinate actions (e.g., lawn mower, web servers, databases, GUI applications).
- Analogy:
  - Think of IPC as "phone calls" or "letters" between processes.

Source: (Mistral AI, 2025)

2) IPC METHODS

- There are many different methods (with pros and cons) available to make processes communicate:
  - Signals
  - Pipes
  - Message queues
  - Shared memory
  - Semaphores
  - Sockets

Source: (Mistral AI, 2025)

3) SIGNALS

1. In the Linux system, signals are an asynchronous notification mechanism used for passing events and information between processes or between the operating system and processes. Processes can respond to external events, such as user input, hardware exceptions, or actions from other processes, through signals.

2. User processes have three ways to respond to signals:

   - Ignore the signal: The process takes no action when the signal is received. However, there are two signals that cannot be ignored, namely SIGKILL and SIGSTOP. SIGKILL is used to immediately terminate the execution of a process, while SIGSTOP is used to pause a process's execution
   - Catch the signal: Define a signal handler function to execute a custom action when the signal is received. Applications can use system calls like signal() or sigaction() to register signal handler functions and respond to specific signals
   - Perform the default action: Linux defines default actions for each type of signal. For example, when a process receives the SIGTERM signal, Linux defaults to terminating the execution of that process. Applications can choose not to define a signal handler function and let the system perform the default action for specific signals

Source: (Luckfox, 2025)

3) SIGNALS

- To send a signal to a process from the terminal, we must use the **kill** instructions, what are the possible signals?

Source: (Luckfox, 2025)

3) SIGNALS

Signal	Signal Value	Default Action	Description
SIGHUP	1	Terminate	Hangup signal, the controlling terminal is closed
SIGINT	2	Terminate	Interrupt signal (Ctrl-C)
SIGQUIT	3	Terminate with core dump	Quit signal (Ctrl-)
SIGILL	4	Terminate with core dump	Illegal instruction
SIGTRAP	5	Terminate with core dump	Trace trap
SIGABRT	6	Terminate with core dump	Abort signal from abort() function
SIGBUS	7	Terminate with core dump	Bus error
SIGFPE	8	Terminate with core dump	Floating-point arithmetic error
SIGKILL	9	Terminate	Kill signal, unblockable

Source: (Luckfox, 2025)

3) SIGNALS

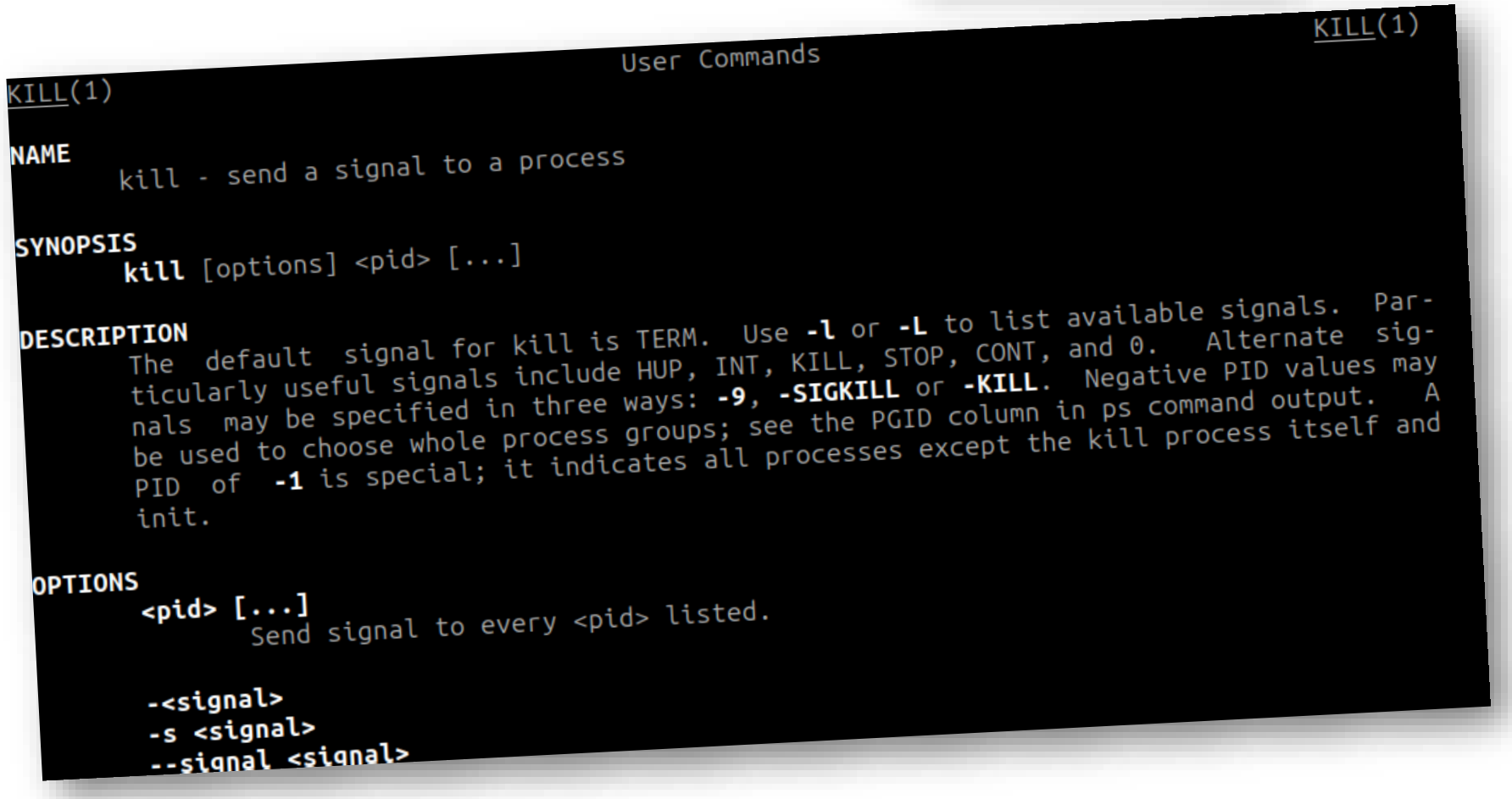
SIGUSR1	10	Terminate	User-defined signal 1
SIGSEGV	11	Terminate with core dump	Invalid memory reference
SIGUSR2	12	Terminate	User-defined signal 2
SIGPIPE	13	Terminate	Write on a pipe with no reader
SIGALRM	14	Terminate	Alarm clock signal
SIGTERM	15	Terminate	Software termination signal, can be caught by the process
SIGSTKFLT	16	Terminate (b)	Stack fault
SIGCHLD	17	Ignore	Child process has terminated
SIGCONT	18	Ignore	Continue executing, if stopped
SIGSTOP	19	Stop	Stop the process

Source: (Luckfox, 2025)

3) SIGNALS

- How to use signals in the terminal?

```
$ man kill
```



KILL(1) User Commands KILL(1)

NAME
kill - send a signal to a process

SYNOPSIS
kill [options] <pid> [...]

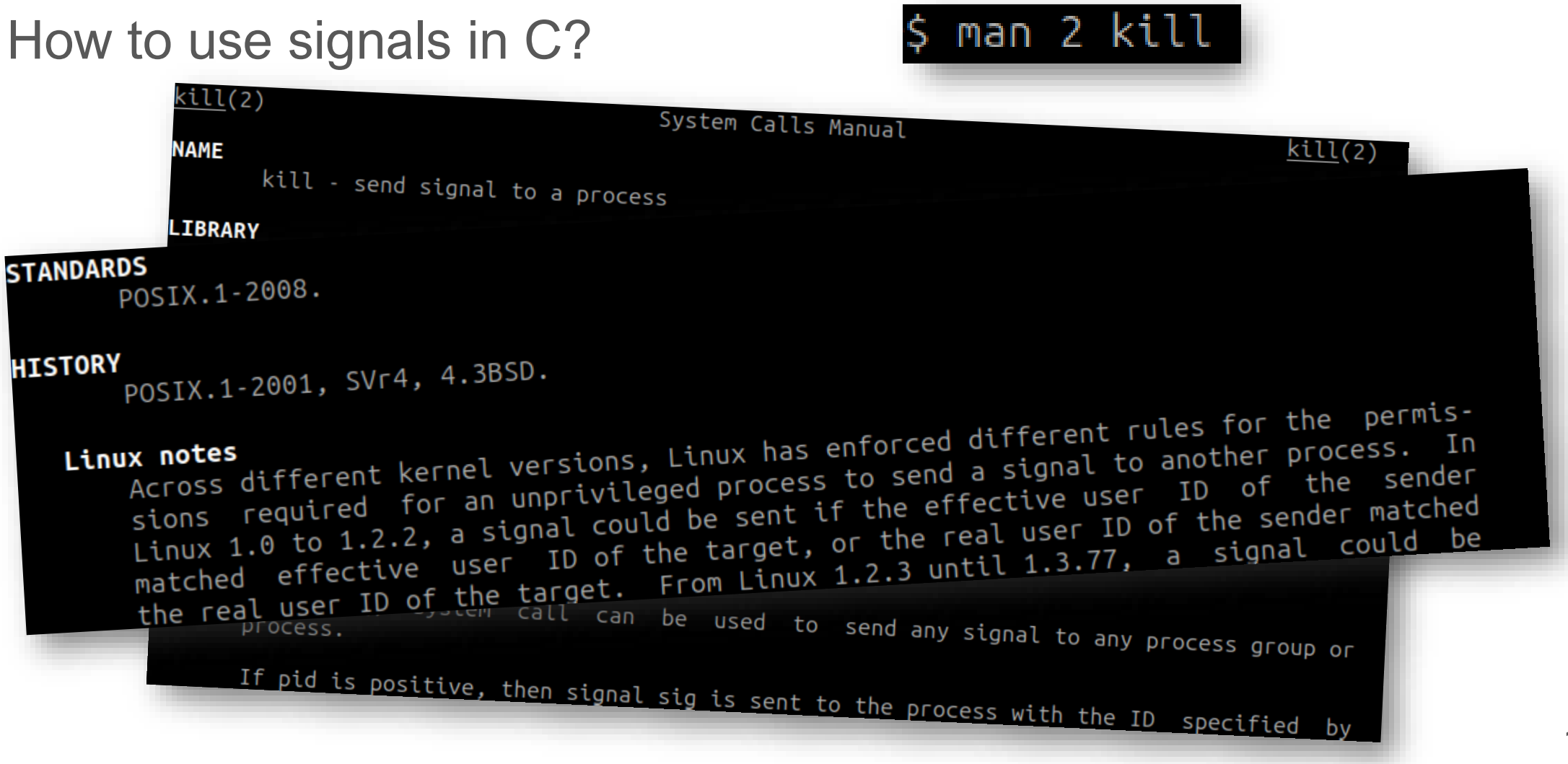
DESCRIPTION
The default signal for kill is TERM. Use **-l** or **-L** to list available signals. Particularly useful signals include HUP, INT, KILL, STOP, CONT, and 0. Alternate signals may be specified in three ways: **-9**, **-SIGKILL** or **-KILL**. Negative PID values may be used to choose whole process groups; see the PGID column in ps command output. A PID of **-1** is special; it indicates all processes except the kill process itself and init.

OPTIONS
<pid> [...]
Send signal to every <pid> listed.

-<signal>
-s <signal>
--signal <signal>

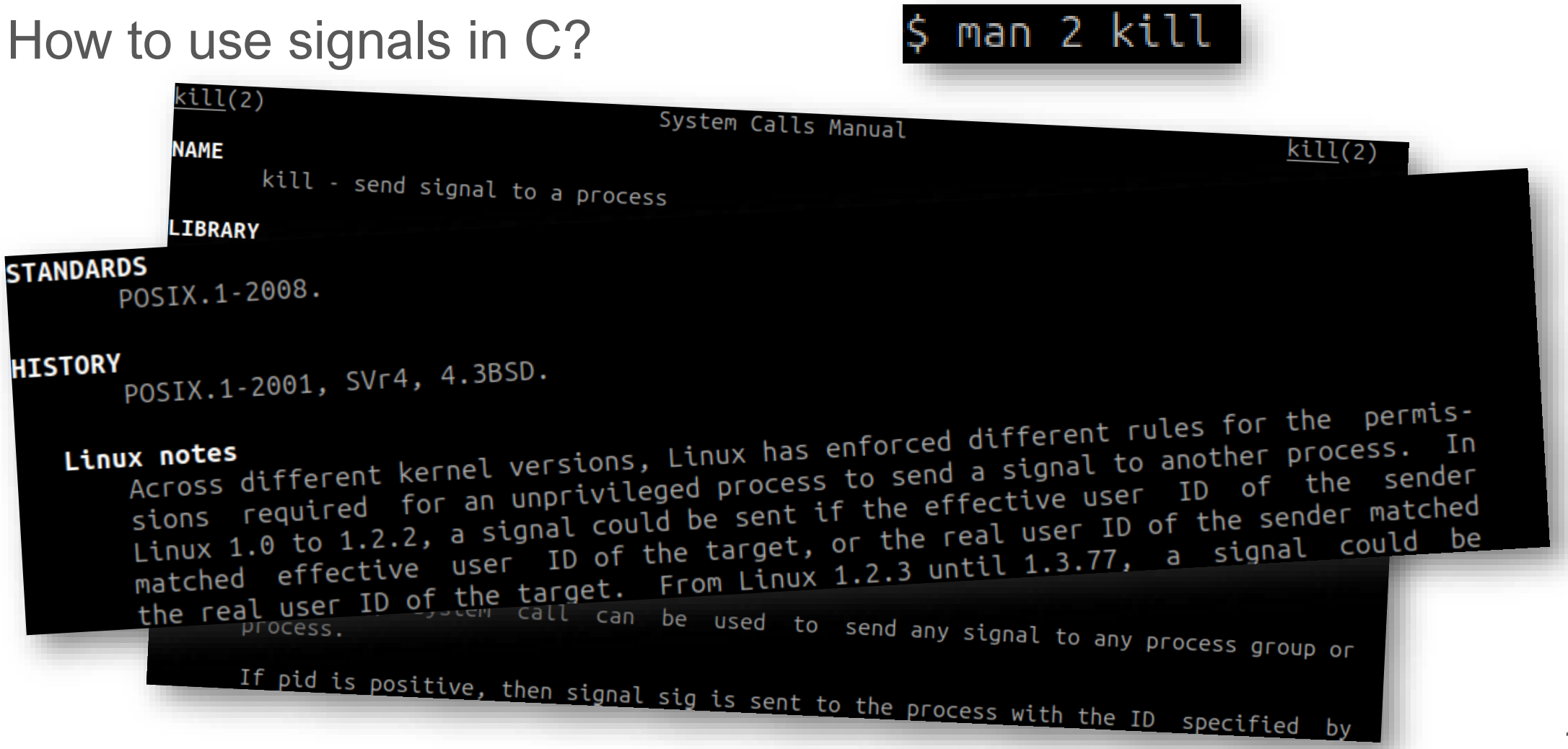
3) SIGNALS

- How to use signals in C?



3) SIGNALS

- How to use signals in C?



```c
#include <signal.h>
#include <stdlib.h>

int main(int argc,char *argv[])
{
    pid_t pid;
    int sig;

    if(argc < 3){
        printf("Usage:%s <pid_t> <signal>\n",argv[0]);
        return -1;
    }
    // Convert strings to integers
    sig = atoi(argv[2]);
    pid = atoi(argv[1]);

    kill(pid,sig);

    return 0;
}
```

```c
#include <stdio.h>
#include <signal.h>

void signal_handler_fun(int signum)
{
    printf("catch signal %d\n", signum);
}

int main(int argc, char *argv[])
{
    signal(SIGINT, signal_handler_fun);
    while (1);
    return 0;
}
```

3) SIGNALS

- **Use Cases**
  - Process control (SIGKILL, SIGSTOP)
  - Exception handling (SIGSEGV, SIGFPE)
  - Simple event notification
  - Timeout implementation (SIGALRM)
- **Pros:**
  - Lightweight and fast, asynchronous notification, built into process management
- **Cons:**
  - **Very limited data** (signal number only, some systems allow small integer), can interrupt execution at any time, limited number of signals, difficult to program correctly (race conditions)

4) PIPES

- Anonymous Pipes: An anonymous pipe is a simple half-duplex pipe that can only be used for communication between processes with a parent-child relationship. Once created, the anonymous pipe becomes shared file descriptors between the two processes, where one process writes data to the pipe and the other process reads data from it. The lifetime of an anonymous pipe is associated with the processes that created it, and when the creating process terminates, the anonymous pipe is destroyed.

- Named Pipes: Named pipes, also known as FIFO (First In, First Out), are a special type of file that allows any process to access it using its filename at any time. It provides a communication mechanism between processes that are not necessarily related and can allow multiple processes to access it simultaneously, making it suitable for interprocess communication over networks. Named pipes persist in the file system until they are deleted or the system is shut down. Any process with permission to access the named pipe can write to and read from it.

```c
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

void sys_err(const char *str)
{
    perror(str);
    exit(1);
}

int main(void)
{
    pid_t pid;
    char buf[1024];
    int fd[2];
    char p[] = "test for pipe\n";
    if (pipe(fd) == -1)
        sys_err("pipe");

    pid = fork();
    if (pid < 0) {
        sys_err("fork err");
    }
    else if (pid == 0) {
        close(fd[1]);
        printf("child process wait to read:\n");
        int len = read(fd[0], buf, sizeof(buf));
        write(STDOUT_FILENO, buf, len);
        close(fd[0]);
    }
    else {
        close(fd[0]);
        write(fd[1], p, strlen(p));
        wait(NULL);
        close(fd[1]);
    }

    return 0;
}
```

Source: (Luckfox, 2025)

4) PIPES

- Anonymous Pipes: Unidirectional, parent-child communication only
- Named Pipes (FIFOs): Bidirectional, unrelated processes can communicate
- Use Cases
  - Shell command chaining (ls | grep myfilename | sort)
  - Simple parent-child data streaming
  - Filter chains in data processing
- Pros: Simple API (read/write like files), automatic synchronization (blocking I/O), kernel-managed buffering
- Cons: Limited to byte streams (no structured data), anonymous pipes require parent-child relationship, unidirectional communication requires two pipes for bidirectionality, no random access (sequential only)

5) MESSAGE QUEUES

In the Linux system, a message queue is an inter-process communication (IPC) mechanism used for data transmission between different processes. It is a first-in-first-out (FIFO) data structure that allows one or multiple processes to communicate by sending and receiving messages in the message queue. The message queue is identified by a message queue identifier (mqid), similar to a file descriptor, which ensures the uniqueness of the message queue. When creating a message queue, a unique key needs to be specified. This key is used to identify the message queue within the system, ensuring that multiple processes can access the same message queue using the same key. Message queues enable data sharing between different processes, making them a powerful inter-process communication mechanism.

Source: (Luckfox, 2025)

5) MESSAGE QUEUES

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MSG_KEY 1234

/* Define the message structure */
struct mymsg {
    long mtype;           /* Message type */
    char mtext[1024];     /* Message content */
};
```

```c
/* Sender process */
void sender()
{
    int msgid, ret;
    struct mymsg msg;

    /* Create or open the message queue */
    msgid = msgget(MSG_KEY, IPC_CREAT | 0666);
    if (msgid < 0) {
        perror("msgget");
        exit(1);
    }

    /* Construct the message */
    msg.mtype = 1;
    strncpy(msg.mtext, "Hello, waveshare!", 1024);

    /* Send the message */
    ret = msgsnd(msgid, &msg, sizeof(struct mymsg) - sizeof(long), 0);
    if (ret < 0) {
        perror("msgsnd");
        exit(1);
    }

    printf("Sent message: %s\n", msg.mtext);
}
```

Source: (Luckfox, 2025)

```c
/* Receiver process */
void receiver()
{
    int msgid, ret;
    struct mymsg msg;

    /* Open the message queue */
    msgid = msgget(MSG_KEY, IPC_CREAT | 0666);
    if (msgid < 0) {
        perror("msgget");
        exit(1);
    }

    /* Receive the message */
    ret = msgrcv(msgid, &msg, sizeof(struct mymsg) - sizeof(long), 1, 0);
    if (ret < 0) {
        perror("msgrcv");
        exit(1);
    }

    printf("Received message: %s\n", msg.mtext);
}
```

Source: (Luckfox, 2025)

5) MESSAGE QUEUES

```c
int main()
{
    pid_t pid;

    /* Create a child process */
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    } else if (pid == 0) {
        /* Child process acts as the sender */
        sender();
    } else {
        /* Parent process acts as the receiver */
        receiver();
    }

    return 0;
}
```

Source: (Luckfox, 2025)

5) MESSAGE QUEUES

- Kernel-managed linked lists of messages with priority and type-based retrieval.
- Use Cases
  - Asynchronous communication between unrelated processes
- Pros:
  - Structured communication, multiple readers/writers supported, non-blocking options available, messages can be retrieved by type/priority
- Cons:
  - System-wide limits on queue size and message count, persistence issues (can survive process termination, requiring cleanup), more complex API than pipes

6) SHARED MEMORY

Shared memory refers to a mechanism where multiple processes share the same physical memory. This mechanism allows multiple processes to access the same memory area, enabling data sharing between processes.

Under the shared memory mechanism, multiple processes can map the same memory region to their address space to achieve data sharing. This allows multiple processes to read and write the same memory, achieving communication and synchronization between them without the need for any data copying or inter-process communication operations. Therefore, shared memory is an efficient inter-process communication mechanism.

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <stdlib.h>

int main(void)
{

    int shmid;
    key_t key;
    pid_t pid;
    char *s_addr, *p_addr;
    key = ftok("./a.c", 'a');
    shmid = shmget(key, 1024, 0777 | IPC_CREAT);
    if (shmid < 0)
    {
        printf("shmget is error\n");
        return -1;
    }
    printf("shmget is ok and shmid is %d\n", shmid);
    pid = fork();
    if (pid > 0)
    {
        p_addr = shmat(shmid, NULL, 0);
        strncpy(p_addr, "hello", 5);
        wait(NULL);
        exit(0);
    }
    if (pid == 0)
    {

        sleep(2);
        s_addr = shmat(shmid, NULL, 0);
        printf("s_addr is %s\n", s_addr);
        exit(0);
    }
    return 0;
}
```

6) SHARED MEMORY

- Multiple processes map the same physical memory region into their address spaces.
- **Use Cases**
  - High-volume data exchange (video streaming, large datasets)
  - Real-time systems requiring minimal latency
  - Database systems and caching servers
- **Pros:**
  - **Fastest IPC method** (no kernel involvement after setup)
  - No data copying between processes
  - Direct memory access with pointer operations
- **Cons:**
  - **No synchronization provided** (requires semaphores/mutexes)
  - Complex programming model (race conditions, memory management)
  - Security concerns (one process can corrupt another's data)
  - Requires careful memory layout planning

7) SEMAPHORES

System V Semaphores are a mechanism for inter-process synchronization and mutual exclusion. They use a counter-based approach to control access to shared resources, allowing multiple processes to access the resource simultaneously while limiting the number of concurrent accesses. This mechanism ensures that each process can smoothly acquire control of the resource, achieving synchronization and mutual exclusion between multiple processes.

```c
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <unistd.h>

union semun
{
    int val;
};

int main(void)
{
    int semid;
    int key;
    pid_t pid;
    struct sembuf sem;
    union semun semun_union;
    key = ftok("./a.c", 0666);
    semid = semget(key, 1, 0666 | IPC_CREAT);
    semun_union.val = 0;
    semctl(semid, 0, SETVAL, semun_union);
    pid = fork();
    if (pid > 0)
    {
        sem.sem_num = 0;
        sem.sem_op = -1;  // P operation
        sem.sem_flg = 0;
        semop(semid, &sem, 1);
        printf("This is parents\n");
        sem.sem_num = 0;
        sem.sem_op = 1;
        sem.sem_flg = 0;
        semop(semid, &sem, 1);
    }

    if (pid == 0)
    {
        sleep(2);
        sem.sem_num = 0;
        sem.sem_op = 1; //V operation
        sem.sem_flg = 0;
        semop(semid, &sem, 1);
        printf("This is son\n");
    }
    return 0;
}
```

7) SEMAPHORES

- Kernel-maintained integer counters for synchronization
- **Use Cases**
  - Controlling access to shared resources
  - Process synchronization and mutual exclusion
  - Producer-consumer problems
  - Often used with shared memory
- **Pros:**
  - Atomic operations guaranteed by kernel
  - Efficient for synchronization
  - Supports multiple resource counting
- **Cons:**
  - Does not transfer data (pure synchronization), deadlock potential if misused, cleanup issues (can persist after process termination), semaphores have complex API

8) SOCKETS

What is a socket in Linux? How to use sockets to make process exchange information, can you produce C code for it?

In Linux (and Unix-like systems), a **socket** is an endpoint for communication between two processes. Sockets allow processes to exchange data, either on the same machine or across a network. They are a fundamental part of inter-process communication (IPC) and network programming.

Types of Sockets

- **Stream Sockets (SOCK_STREAM):** Provide reliable, connection-oriented communication (e.g., TCP).
- **Datagram Sockets (SOCK_DGRAM):** Provide connectionless, unreliable communication (e.g., UDP).
- **Raw Sockets (SOCK_RAW):** Allow direct access to lower-level protocols.

Source: (Mistral, 2025)

8) SOCKETS

How to Use Sockets for Process Communication

To demonstrate, I'll provide a simple C code example for **local communication** (between processes on the same machine) using **Unix domain sockets** (also called IPC sockets). This example consists of a **server** and a **client** process.

Example: Unix Domain Socket Communication

Server Code

The server creates a socket, binds it to a file, listens for connections, and accepts a client connection.

Source: (Mistral, 2025)

8) SOCKETS

Server Code

The server creates a socket, binds it to a file, listens for connections, and accepts a client connection.

C Copy

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <string.h>

#define SOCKET_PATH "/tmp/mysocket"

int main() {
```

Source: (Mistral, 2025)

8) SOCKETS

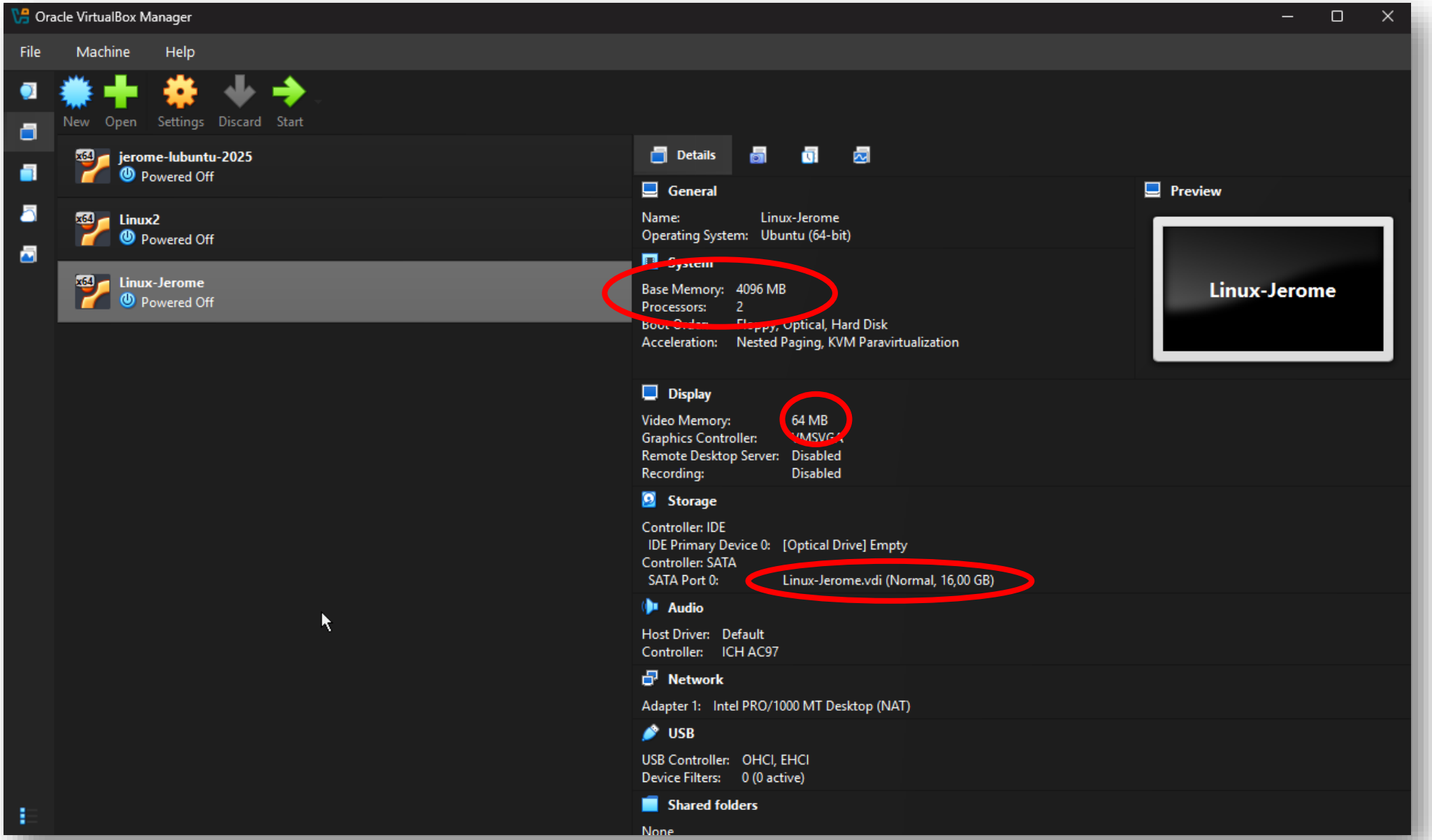
Client Code

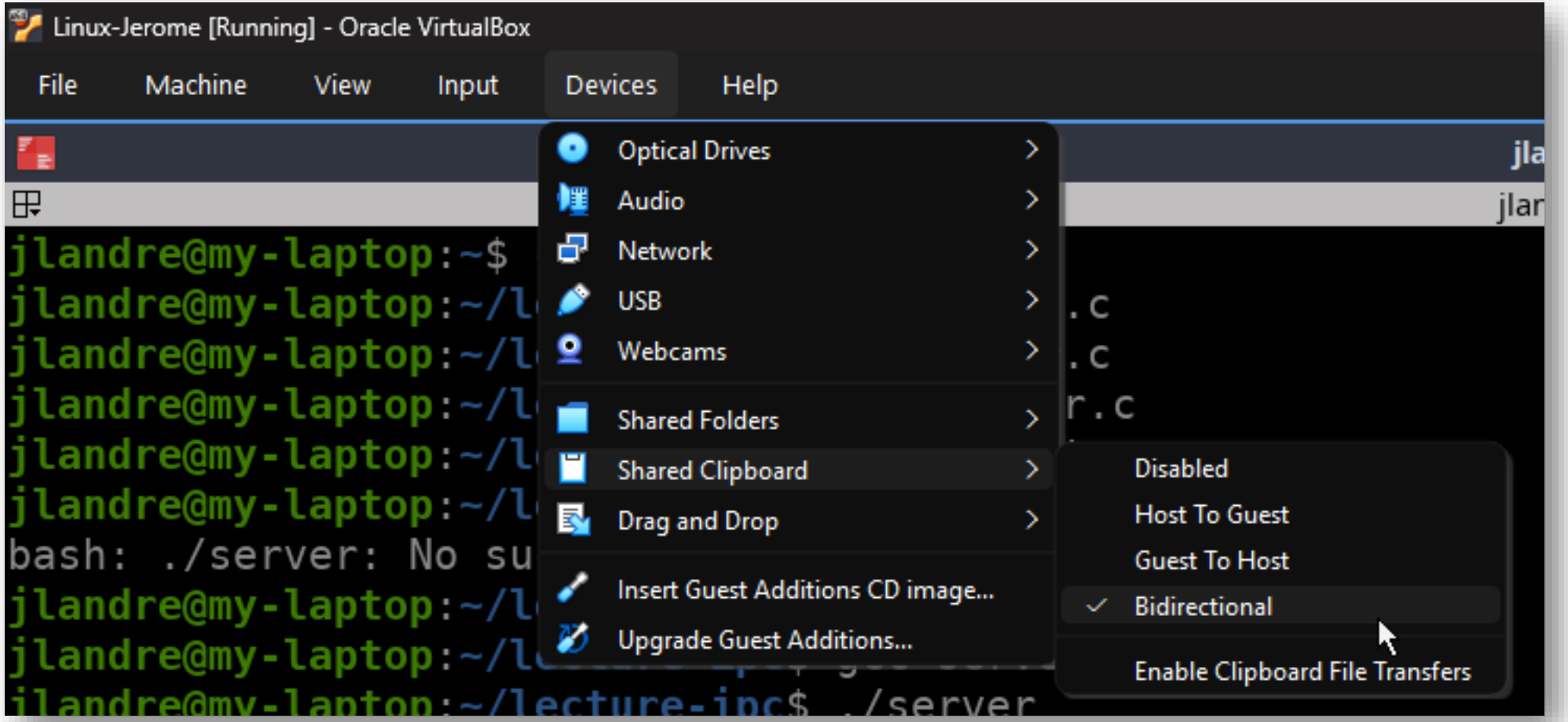
The client connects to the server socket and sends a message.

C Copy

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <string.h>

#define SOCKET_PATH "/tmp/mysocket"

int main() {
    int client_fd;
```

Source: (Mistral, 2025)


Oracle VirtualBox Manager
File
Machine
Help
New
Open
Settings
Discard
Start
jerome-lubuntu-2025
Powered Off
Linux2
Powered Off
Linux-Jerome
Powered Off
Details
General
Name: Linux-Jerome
Operating System: Ubuntu (64-bit)
Base Memory: 4096 MB
Processors: 2
Acceleration: Nested Paging, KVM Paravirtualization
Preview
Linux-Jerome
Display
Video Memory: 64 MB
Remote Desktop Server: Disabled
Recording: Disabled
Storage
Controller: IDE
IDE Primary Device 0: [Optical Drive] Empty
Controller: SATA
SATA Port 0: Linux-Jerome.vdi (Normal, 16,00 GB)
Audio
Host Driver: Default
Controller: ICH AC97
Network
Adapter 1: Intel PRO/1000 MT Desktop (NAT)
USB
USB Controller: OHCI, EHCI
Device Filters: 0 (0 active)
Shared folders
None


Linux-Jerome [Running] - Oracle VirtualBox
File
Machine
View
Input
Devices
Help
Optical Drives
Audio
Network
USB
Webcams
Shared Folders
Shared Clipboard
Drag and Drop
Insert Guest Additions CD image...
Upgrade Guest Additions...
Disabled
Host To Guest
Guest To Host
Bidirectional
Enable Clipboard File Transfers
jlandre@my-laptop:~$
bash: ./server: No su
jlandre@my-laptop:~/lecture-ipc$ ./server

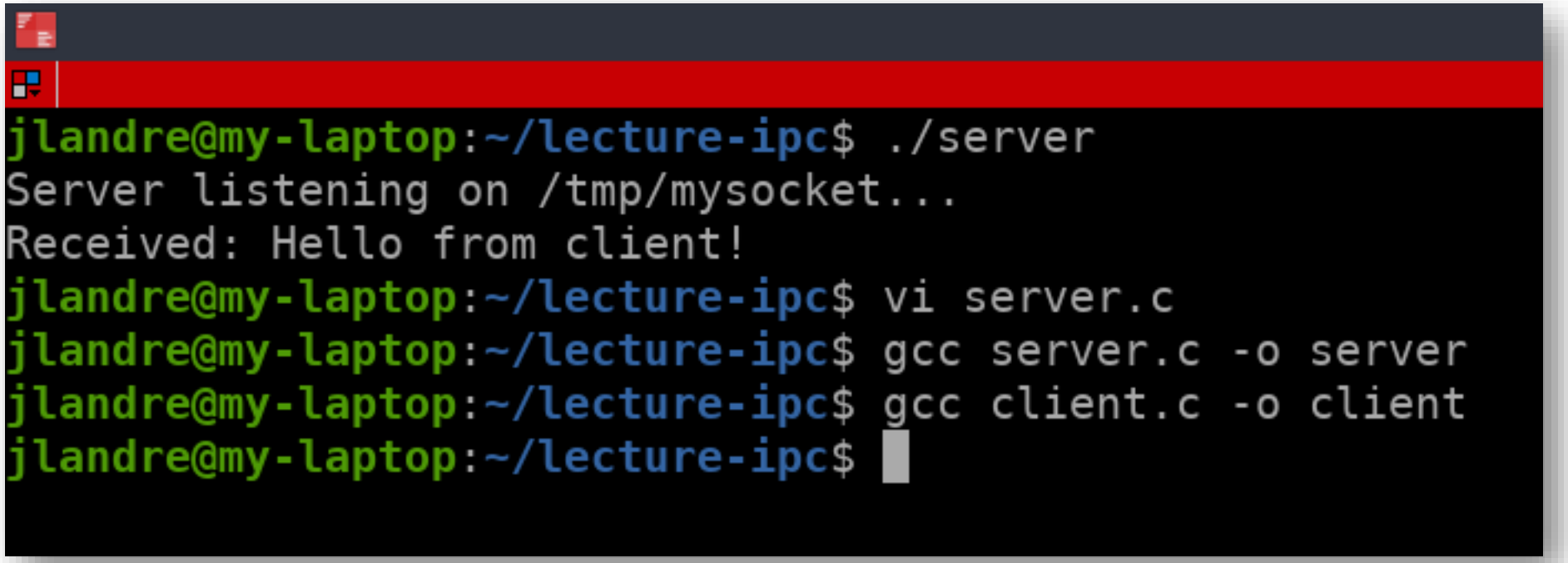
Linux-Jerome [Running] - Oracle VirtualBox

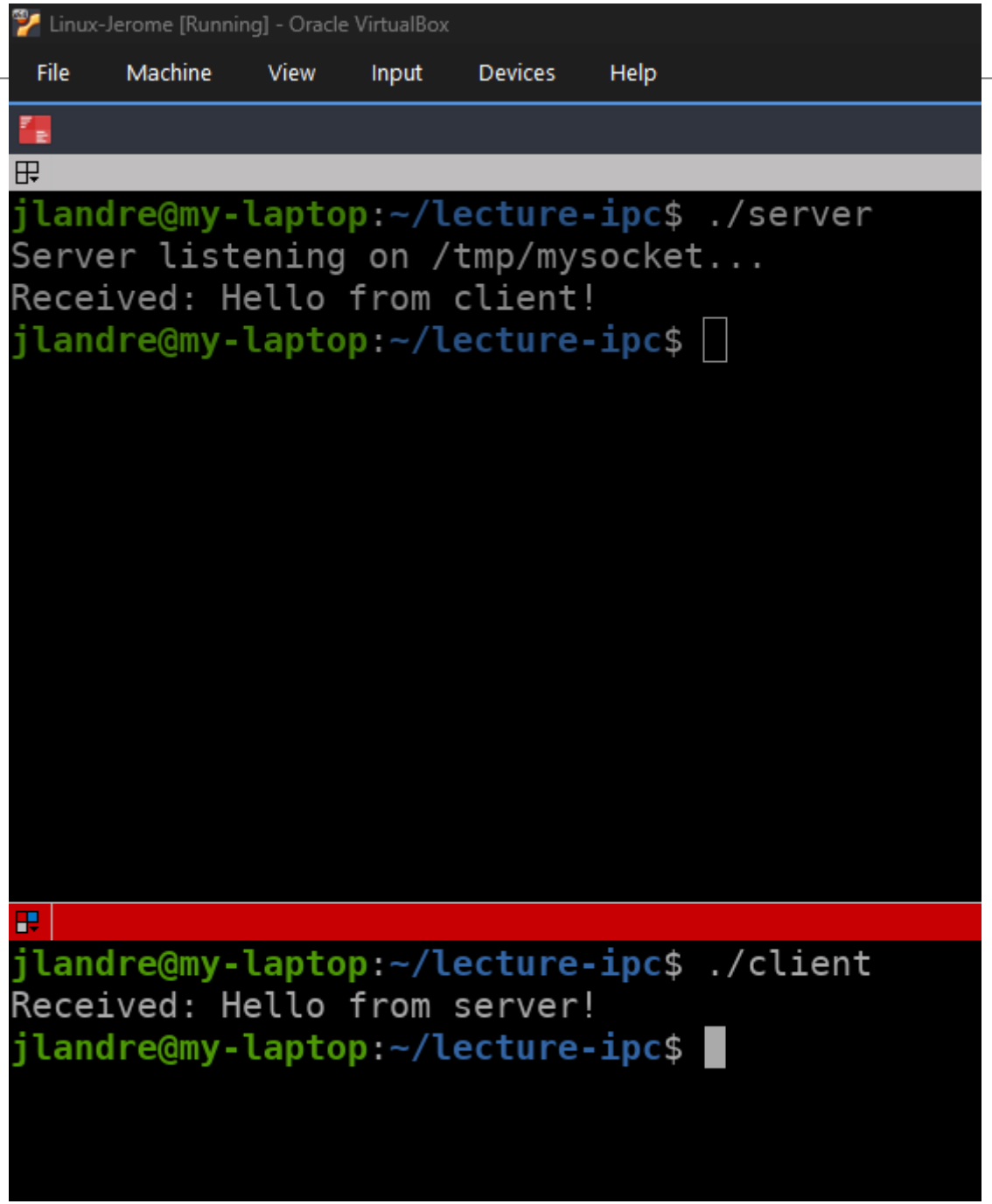
File Machine View Input Devices Help

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <string.h>

#define SOCKET_PATH "/tmp/mysocket"

int main() {
    int server_fd, client_fd;
    struct sockaddr_un server_addr, client_addr;
    socklen_t client_len;
    char buffer[256];
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <string.h>

#define SOCKET_PATH "/tmp/mysocket"

int main() {
    int client_fd;
    struct sockaddr_un server_addr;
    char buffer[256];

    // Create socket
    client_fd = socket(AF_UNIX, SOCK_STREAM, 0);
"client.c" 48L, 1147B
```

1 2 3 4 jlandre@my-laptop: ~...


jlandre@my-laptop:~/lecture-ipc$ ./server
Server listening on /tmp/mysocket...
Received: Hello from client!
jlandre@my-laptop:~/lecture-ipc$ vi server.c
jlandre@my-laptop:~/lecture-ipc$ gcc server.c -o server
jlandre@my-laptop:~/lecture-ipc$ gcc client.c -o client
jlandre@my-laptop:~/lecture-ipc$


Linux-Jerome [Running] - Oracle VirtualBox
File
Machine
View
Input
Devices
Help
jlandre@my-laptop:~/lecture-ipc$ ./server
Server listening on /tmp/mysocket...
Received: Hello from client!
jlandre@my-laptop:~/lecture-ipc$
jlandre@my-laptop:~/lecture-ipc$ ./client
Received: Hello from server!
jlandre@my-laptop:~/lecture-ipc$

JL Can you generate a simple C code example to create a process, a child and create a network socket on the local machine to make them exchange the texts "Hi, how are You?" and "well, thank you and you?"?

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/wait.h>

#define SOCKET_PATH "/tmp/ipc_socket"
#define BUFFER_SIZE 256

void child_process() {
    int sock;
    struct sockaddr_un addr;
    char buffer[BUFFER_SIZE];

    // Wait a bit for parent to setup server
    sleep(1);

    // Create socket
    sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock == -1) {
        perror("Child: socket creation failed");
        exit(1);
    }

    // Setup address structure
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
```

Source: (Anthropic, 2025)

I've created a complete C example demonstrating IPC using Unix Domain Sockets. Here's what the code does:

Key Components:

1. **Parent Process (Server):**
   - Creates a Unix domain socket
   - Binds it to `/tmp/ipc_socket`
   - Listens for connections
   - Sends "Hi, how are You?" to child
   - Receives response from child
2. **Child Process (Client):**
   - Connects to the parent's socket
   - Receives the greeting message
   - Sends back "well, thank you and you?"

Compilation and Execution:

bash

```
gcc -o socket_ipc socket_ipc.c
./socket_ipc
```

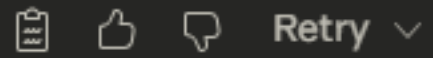
Expected Output:

```
Starting IPC demonstration using Unix Domain Sockets
================================================

PARENT: Waiting for child connection...
CHILD: Connected to parent
PARENT: Child connected
PARENT: Sent: "Hi, how are You?"
CHILD: Received: "Hi, how are You?"
CHILD: Sent: "well, thank you and you?"
PARENT: Received: "well, thank you and you?"

IPC completed successfully!
```

Educational Points:

- **AF_UNIX**: Unix domain socket (local machine only)
- **SOCK_STREAM**: Connection-oriented (like TCP)
- **fork()**: Creates the child process
- **Synchronization**: Child sleeps briefly to let parent setup server
- **Cleanup**: Socket file is removed after use

This is ideal for teaching because it shows the complete socket lifecycle: create → bind → listen → accept for server, and create → connect for client!



Claude can make mistakes. Please double-check responses.

8) SOCKETS

- Sockets: Local machine IPC and Network TCP/UDP distributed systems
- **Use Cases**
  - Client-server architectures
  - Network communication
  - Bidirectional full-duplex communication
  - Cross-platform IPC
- **Pros:**
  - Versatile (local and network communication)
  - Well-established protocols and tools
  - Bidirectional, full-duplex communication
  - Language and platform independent
- **Cons:**
  - Complex API and setup, network sockets involve protocol overhead, error handling complexity (network issues, timeouts)

REFERENCES

- Luckfox. (2025). *Inter-Process-Communication*. https://wiki.luckfox.com/Core3566/Linux-Systems-Programming/4.Inter-process-communication
- Mistral AI. (2025). *Le Chat*. https://mistral.ai/
- Anthropic. (2025). *Claude Sonnet 4.5* [Large language model]. https://claude.ai

ANY QUESTIONS?